

0_Python_wprowadzenie

March 3, 2022

1 Zmienne

```
[ ]: # Wyświetlanie tekstu  
print("Hello world")
```

```
[ ]: # Tworzenie zmiennych  
# Liczba całkowita  
zmienna = 2  
  
# Liczba rzeczywista  
rzeczywista = 4.5  
rzeczywista = float(34)  
rzeczywista = 34.0  
  
# typu string  
zmiennaTekstowa1 = "jakis tekst"  
zmiennaTekstowa2 = "to jest nowa zmienna"  
  
# Łączenie i wyświetlanie tekstu  
print(zmiennaTekstowa1 + " " + zmiennaTekstowa2)
```

```
[ ]: # Nie można bezpośrednio łączyć zmiennych tekstowych i liczbowych  
print(zmienna + zmiennaTekstowa1)
```

```
[ ]: # Operacje na obiektach  
# Dostępne są podobnie jak w innych językach operatory:  
# dodawania(+), odejmowania(-), mnożenia(*), dzielenia(/), operacji modulo(%)  
# Podnoszenie do potęgi  
kwadrat = 3**2  
szescian = 6**3
```

2 Tablice

```
[ ]: # Tworzenie tablic
tablica = []
# dodawanie nowych wartości do tablicy
tablica.append(23.2)
tablica.append(12.3)
tablica.append(4)
```

```
[ ]: # Sprawdzenie typu zmiennej
print(type(zmienna))
print(type(rzeczywista))
print(type(zmiennaTekstowa1))
print(type(tablica))
```

```
[ ]: # Sprawdzenie długości tablicy
print(len(tablica))

# Wyświetlanie kolejnych wartości z tablicy
print("Kolejne wartości elementów tablicy:")
print(tablica[0])
print(tablica[1])
print(tablica[2])
```

```
[ ]: # Inny sposób na wyświetlenie kolejnych elementów tablicy poprzez konwersję
    ↪ elementów na string
print("Kolejne elementy tablicy: " + str(tablica[0]) + ", " + str(tablica[1]) +
    ↪ ", " + str(tablica[2]))

# Wyświetlenie elementu i jego typu
print("Liczba: " + str(tablica[0]) + " typ " + str(type(tablica[0])))
print("Liczba: " + str(tablica[1]) + " typ " + str(type(tablica[1])))
print("Liczba: " + str(tablica[2]) + " typ " + str(type(tablica[2])))
```

3 Formatowanie tekstu i obsługa napisów

```
[ ]: # Łączenie napisów
tekst = "witaj" + " " + "swiecie"
```

```
[ ]: # Operacja "mnożenia" napisów pozwala na wygenerowanie powtarzających się
    ↪ sekwencji danego napisu
witaj = "witaj " * 10
print(witaj)
```

```
[ ]: print("Tablice o powtarzającym się ciągu elementów")
print([4,5] * 4)
```

```
[ ]: # Podstawowe specyfikatory do formatowania napisów
# %s - napis lub każdy inny obiekt mający reprezentację w formie napisu
# %d - liczby całkowite
# %f - liczby rzeczywiste
# %.<ilosc cyfr> - liczba zmiennie przecinkowa ze stałą liczbą cyfr po kropce

# Dla większej liczby wyświetlanych zmiennych, zmienne powinny znaleźć się po
→znaku % w nawiasie
print("Kolejne elementy tablicy: %f, %.1f, %d" % (tablica[0], tablica[1],
→tablica[2]))
print("Elementy tablicy %f oraz jego typ %s" % (tablica[0], type(tablica[0])))
print("Elementy tablicy {:f} oraz jego typ {}".format(tablica[0],
→type(tablica[0])))
print()
# Dla pojedynczej zmiennej nie ma konieczności umieszczania jej w nawiasie
print("Element tablicy %f" % tablica[0])
print("Element tablicy {:f}".format(tablica[0]))
print()
# Ograniczenie wyświetlania liczb znajdujących się po przecinku przy wyświetlaniu
print("Element tablicy %.1f" % tablica[0])
print("Element tablicy {:.1f}".format(tablica[0]))
```

```
[ ]: # Sprawdzenie liczby przechowywanych znaków w zmiennej tekstowej. Podobnie jak
→dla tablic wykorzystywana jest
# funkcja len(nazwa_zmiennej)
tekst = "abcdabeg cdadbcn"
print(len(tekst))

# zwrócenie pierwszego występującego indeksu danego znaku lub ciągu znaków
index = tekst.index("bc")
print("index: %d" % index)
```

```
[ ]: # Zliczenie liczby występujących znaków lub ciąg znaków
print("Liczba występujących ciągów znaków 'bc': %s" % tekst.count("bc"))
```

```
[ ]: # Dostęp do pojedynczego znaku
print(tekst[1])
```

```
[ ]: # Dostęp do fragmentu tekstu znajdującego się pomiędzy wskazanymi indeksami
print(tekst[2:5])
```

```
[ ]: # Wyświetlenie całego napisu od początku do wskazanego indeksu znaku
print(tekst[0:5])
print(tekst[:5])
```

```
[ ]: # Wyświetlenie całego napisu od wskazanego znaku do końca tekstu
print(tekst[5:len(tekst)])
print(tekst[5:])

[ ]: # Można również liczyć i wyświetlać znaki od końca. W tym celu podaje się ujemne
    ↪ wartości indeksów
print(tekst[-1])    # Wyświetlenie ostatniego znaku
print(tekst[-5:-3]) # Wyświetlenie znaków pomiędzy 5 (włącznie) pozycją od
    ↪ końca a 3 (włącznie)
print(tekst[-5:])    # Wyświetlenie znaków od 5 (włącznie) pozycji od końca do
    ↪ samego końca
print(tekst[: -5])    # Wyświetlenie wszystkich znaków od początku do 5 znaku od
    ↪ końca (pomijany jest 5 znak od końca)

[ ]: # Zamiana wszystkich liter na duże lub małe
print(tekst.upper())
print(tekst.lower())

[ ]: # Sprawdzenie czy napis zaczyna się lub kończy od podanego ciągu znaków
print(tekst.startswith("cda"))
print(tekst.startswith("abc"))
print(tekst.endswith("abeg"))
print(tekst.endswith("dbcn"))

[ ]: # Rozdzielanie napisu i zapisywanie go w tablicy np. w poniższym przykładzie
    ↪ spacja
# odpowiada za rozdzielenie tekstu
tablica_rozdzielona = tekst.split(" ")
print(tablica_rozdzielona)
```

4 Instrukcje warunkowe

```
[ ]: a = 4
b = 7
print("a == 4 " + str(a == 4)) # True
print("a != 4 " + str(a != 4)) # False
print("a == 5 " + str(a == 5)) # False
print("a > 1 " + str(a > 1)) # True
#zmiana wartości wyrażenia na przeciwną
print("not a == 4 " + str(not a == 4))

[ ]: # Dostępne są również operatory logiczne and, or
print("a == 4 and b > 3 " + str(a == 4 and b > 3)) #True
print("a == 2 or b > 2 " + str(a == 2 or b > 2)) #True
```

```
[ ]: # Do sprawdzenia czy podany operator znajduje się w tablicy służy operator in
tablica = ["abc", "def", "ghi"]
print("def" in tablica) # true
print("test" in tablica) # false
```

```
[ ]: # Do sprawdzenia czy zmienne wskazują na ten sam obszar w pamięci służy operator
↳ is
x = [5, 3, 3]
y = [2, 6, 3]

tablica2 = [4, x, y]

print("x == tablica2[1] " + str(x == tablica2[1])) # True
print("x is tablica2[1] " + str(x is tablica2[1])) # True
print("x is tablica2 " + str(x is tablica2)) # False
```

```
[ ]: # Instrukcja warunkowa if - składnia
if wyrażenie:
    jakas_akcja
elif wyrażenie2:
    jakas_inna_akcja
else:
    jakas_jeszcze_inna_akcja
```

```
[ ]: # Przykład użycia
a = 3
if a == 5:
    print("zmienna a jest równa 5")
elif a > 5:
    print("zmienna a jest większa od 5")
else:
    print("zmienna a jest mniejsza od 5")
```

5 Pętle

```
[ ]: # for - pętla wykonuje się przez określoną liczbę iteracji
liczby = [4, 2, 6, 4, 6]

for liczba in liczby:
    # liczba - zmienna w której przechowywany jest każdy kolejny element tablicy
    ↳ liczby
    # pętla wykonuje się dopóki nie zostaną wyświetlone wszystkie elementy z tablicy
    print(liczba)
```

```
[ ]: # Liczby z wykorzystaniem instrukcji xrange mogą również zawaierać się w
    ↪ określonym przedziale
for i in range(5):
    print(i)
```

```
[ ]: # while - pętla wykonuje się dopóki pewien warunek jest spełniony
i = 0
while i < 5:
    print(i)
    i += 1
```

```
[ ]: # instrukcja break używana jest do zakończenia działania pętli for i while
i = 0
while True:
    print(i) # , powoduje, że kolejne elementy wyświetlane są dalej w linii, a
    ↪ nie nowym wierszu
    i += 1
    if i >= 3:
        break

print(i)
```

```
[ ]: # instrukcja continue odpowiada za opuszczenie instrukcji znajdujących się
    ↪ poniżej i wraca do wykonywania
# kodu znajdującego się od początku pętli
# Poniższa instrukcja wypisze tylko liczby parzyste
for x in range(30):
    # Sprawdź, czy x jest nieparzyste
    if x % 2 != 0:
        continue
    print(x)
```

6 Funkcje

```
[ ]: # Składnia
def nazwa_funkcji():
    kod programu

def nazwa_funkcji(argument1, argument2, ...):
    kod programu

def nazwa_funkcji():
    kod programu
    return wartosc #funkcja zwraca jakąś wartość zapisaną w kodzie programu
```

```
[ ]: # Przykładowa funkcja wyświetlająca informację czy dana liczba jest podzielna
      ↪przez 3
def podzielna_przez_3(liczba):
    if (liczba % 3) == 0:
        print("Podana liczba: {:d}, jest podzielna przez 3".format(liczba))
    else:
        print("Podana liczba: {:d}, nie jest podzielna przez 3".format(liczba))

[ ]: # Wywołanie funkcji
podzielna_przez_3(6)
podzielna_przez_3(2)
```

7 Importowanie modułów

```
[ ]: # importowanie biblioteki
import math
```

```
[ ]: # Wyświetlenie wszystkich dostępnych funkcji dla modułu math
dir(math)
```

```
[ ]: # Wyświetlanie pomocy do funkcji z biblioteki
help(math.log)
```

```
[ ]: class Robot:
    def __init__(self, name, x, y):
        # inicjalizacja klasy
        self.name = name
        self.x = x
        self.y = y

    def print_pose_info(self):
        print("Pose x={:f}, y={:f}".format(self.x, self.y))

    def move_right(self):
        self.x += 1

    def move_left(self):
        self.x -= 1

    def move_up(self):
        self.y += 1

    def move_down(self):
        self.y -= 1
```

```
def get_distance_to_start(self):  
    # metoda zwraca odległość robota od startu (0,0)  
    return math.sqrt(self.x*self.x + self.y+self.y)
```

```
[ ]: # Inicjalizacja klas Robot  
robot1 = Robot("r1", 0, 0)  
robot2 = Robot("r2", 5, 2)
```

```
[ ]: # Odczyt wartości atrybutu klasy  
robot2.name
```

```
[ ]: # Użycie metody klasy  
robot2.move_right()  
robot2.print_pose_info()
```

```
[ ]: # Użycie metody zwracającej jakąś wartość  
dist_to_start= robot2.get_distance_to_start()  
print("Distance to start: ", dist_to_start)
```

```
[ ]: robot1.print_pose_info()  
robot1.move_right()  
robot1.print_pose_info()
```

```
[ ]:
```

```
[ ]:
```